Immutable Booking Receipt & Nightly Settlement

Practice Problem 5

Concept and approach

The class is final and all stored state is final. Both constructor and getter copy arrays defensively. withUpdatedSeat returns a new receipt. GroupBookingReceipt must extend BookingReceipt, so BookingReceipt cannot itself be final; the code makes the concrete RegularBookingReceipt final while keeping the extensible base immutable. Null batch elements are skipped in one pass.

Important implementation notes

The practice prompt asks BookingReceipt to be final and also to have GroupBookingReceipt extend it. Java forbids subclassing a final class. This solution preserves the immutable parent pattern and subclass requirement by leaving BookingReceipt extensible; if finality is mandatory, use a separate immutable interface/record hierarchy rather than extends.

Java solution

```
import java.util.Arrays;

class BookingReceipt {
private final String bookingId;
private final String[] seatNumbers;

public BookingReceipt(String bookingId, String[] seatNumbers) {
if (bookingId == null || seatNumbers == null) throw new IllegalArgumentException();
this.bookingId = bookingId;
this.seatNumbers = Arrays.copyOf(seatNumbers, seatNumbers.length);
}

public String getBookingId() { return bookingId; }
public String[] getSeatNumbers() { return Arrays.copyOf(seatNumbers, seatNumbers.length); }

public BookingReceipt withUpdatedSeat(int index, String newSeat) {
if (index < 0 || index >= seatNumbers.length) throw new IndexOutOfBoundsException();
String[] copy = getSeatNumbers();
copy[index] = newSeat;
return new BookingReceipt(bookingId, copy);
}
}

final class RegularBookingReceipt extends BookingReceipt {
public RegularBookingReceipt(String bookingId, String[] seats) { super(bookingId, seats); }
}

final class GroupBookingReceipt extends BookingReceipt {
private final int groupSize;
public GroupBookingReceipt(String bookingId, String[] seats, int groupSize) {
super(bookingId, seats);
if (groupSize <= 0) throw new IllegalArgumentException("groupSize must be positive");
this.groupSize = groupSize;
}
public int getGroupSize() { return groupSize; }
}

class BookingSettlement {
static String processNightlySettlement(BookingReceipt[] receipts) {
int processed = 0, nullSkipped = 0, groups = 0, individuals = 0;
for (BookingReceipt receipt : receipts) {
if (receipt == null) { nullSkipped++; continue; }
processed++;
if (receipt instanceof GroupBookingReceipt) groups++;
else individuals++;
}
return processed + " processed | " + nullSkipped + " null skipped | " +
groups + " group | " + individuals + " individual";
}
}
```

Sample output

```
2 processed | 1 null skipped | 1 group | 1 individual
```